

String Classes

1.String

- A string can be defined as sequence of characters or numbers or special characters within the pair of double quotes is called as String.
- There are four string classes defined by java
 1. String
 2. String Buffer
 3. String Builder
 4. String Tokenizer

String Classes

All string classes are called as predefined classes or user defined classes.

String class is part of package called java.lang.

To create string-based objects there are two objects

1. Without new keyword
2. With new keyword

Without new keyword

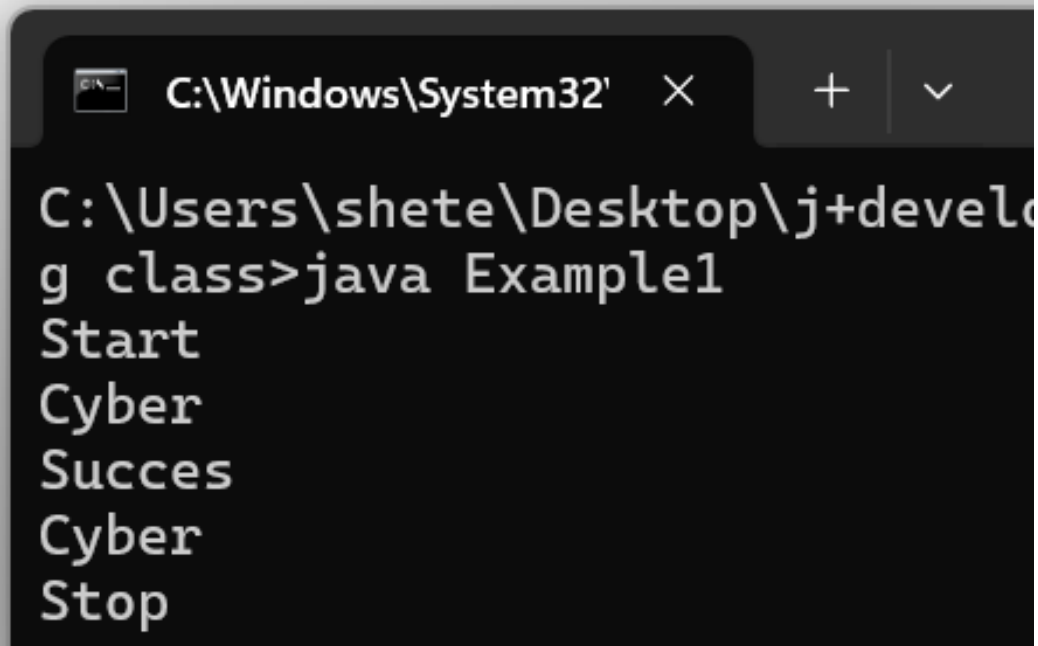
Whenever we create String-based objects without new keyword those objects will get created within the heap memory inside the string constant pool before creating any object within the string constant pool it will check if a object is present or not.

If the object is not present within the string constant pool only then an object gets created within the string constant. But the string object is already present within the string constant pool then java will not create a new object and it will redirect the reference to the existing objects.

7. String Classes

```
class Example1
{
    public static void main (String [] args)
    {
        String str1 = "Cyber";
        String str2 = "Succes";
        String str3 = "Cyber";

        System.out.println("Start");
        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);
        System.out.println("stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt displays the following text:

```
C:\Users\shete\Desktop\j+develo
g class>java Example1
Start
Cyber
Succes
Cyber
Stop
```

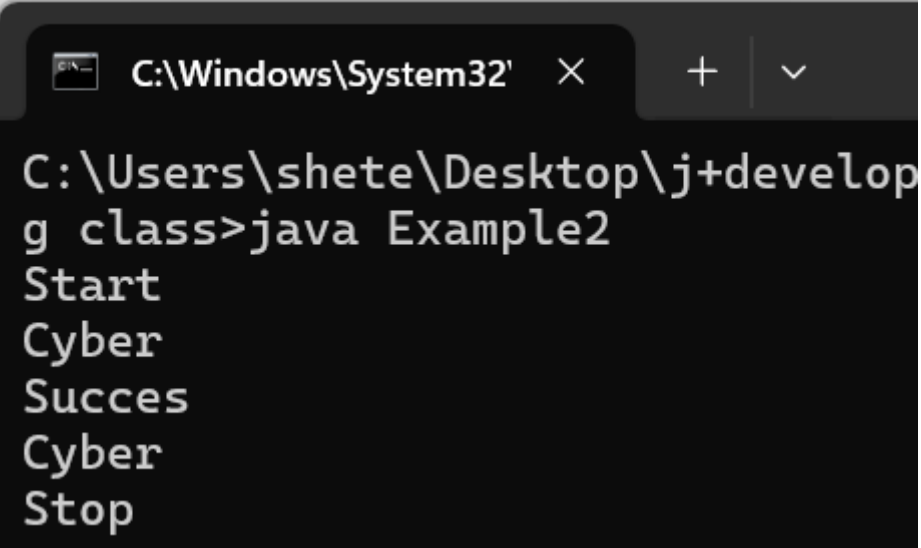
7. String Classes

With new keyword

Whenever we create string-based objects with new keyword these objects will be created as part of heap memory and regardless whether the string is duplicate or not, separate objects will get created in the heap.

```
class Example2
{
    public static void main (String [] args)
    {
        String s1 = new String ("Cyber");
        String s2 = new String ("Succes");
        String s3 = new String("Cyber");

        System.out.println("Start");
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);
        System.out.println("Stop");
    }
}
```



The screenshot shows a Windows command prompt window with a single tab titled 'C:\Windows\System32'. The command prompt displays the following text:

```
C:\Users\shete\Desktop\j+develop
g class>java Example2
Start
Cyber
Succes
Cyber
Stop
```

7. String Classes

To compare the strings we use '==' operator

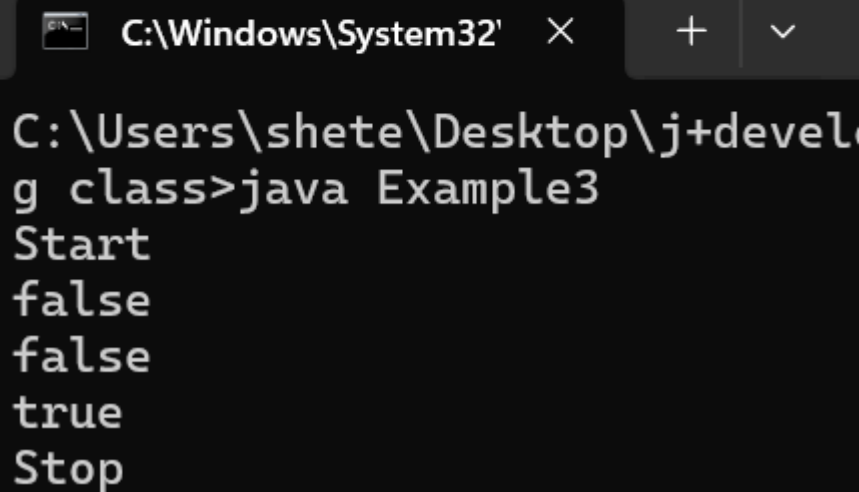
The == operator will perform what comparison

A. Reference comparison

B. Content comparison

```
class Example3
{
    public static void main (String [] args)
    {
        String str1 = "Cyber";
        String str2 = "Succes";
        String str3 = "Cyber";

        System.out.println("Start");
        System.out.println(str1==str2);
        System.out.println(str2==str3);
        System.out.println(str3==str1);
        System.out.println("Stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt shows the following output:

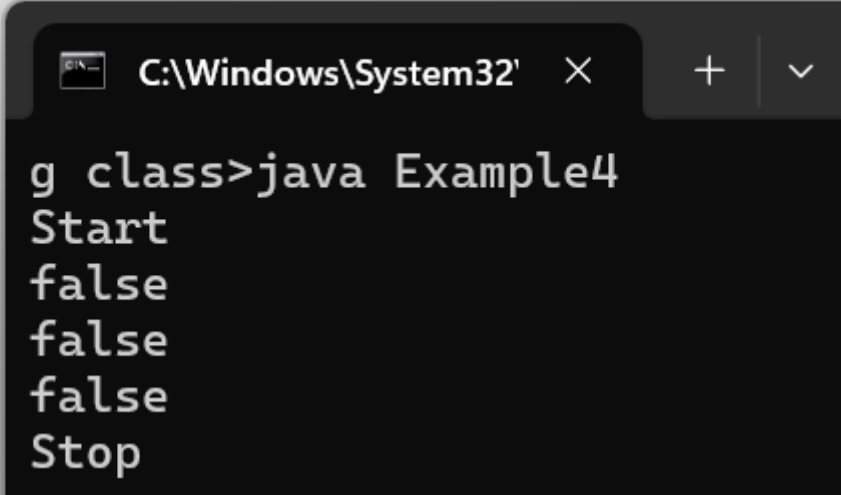
```
C:\Users\shete\Desktop\j+develo
g class>java Example3
Start
false
false
true
Stop
```

7. String Classes

- The == operator will always perform reference comparison.

```
class Example4
{
    public static void main (String [] args)
    {
        String s1 = new String ("Cyber");
        String s2 = new String ("Succes");
        String s3 = new String("Cyber");

        System.out.println("Start");
        System.out.println(s1==s2);
        System.out.println(s2==s3);
        System.out.println(s3==s1);
        System.out.println("Stop");
    }
}
```



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32' and standard window controls. The command prompt displays the command 'g class>java Example4' and its output: 'Start', 'false', 'false', 'false', and 'Stop'.

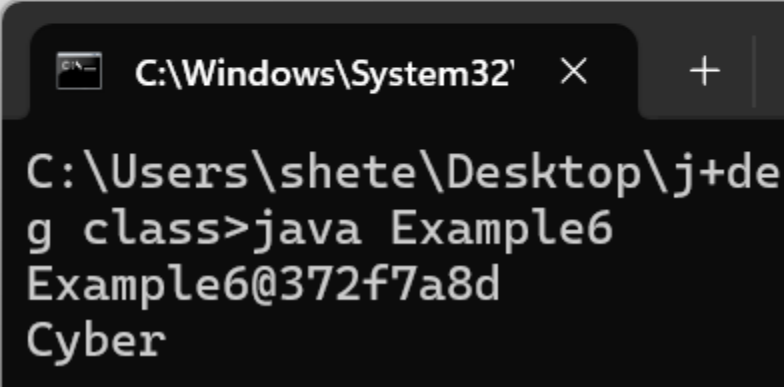
```
g class>java Example4
Start
false
false
false
Stop
```

7. String Classes

Whenever we print a normal class reference variable it will print the address but whenever we print string class reference variable it doesn't print the address but it prints the value, Why?

- Whenever we print the reference variable of normal class it will internally call the toString() of the object class, the toString() of the object class will always be responsible to print the address.
- But within the string class the toString() method is overridden hence rather than returning the address it returns the value.

```
class Example6
{
    public static void main (String [] args)
    {
        Example6 ref = new Example6();
        System.out.println(ref);
        String str = new String("Cyber");
        System.out.println(str);
    }
}
```

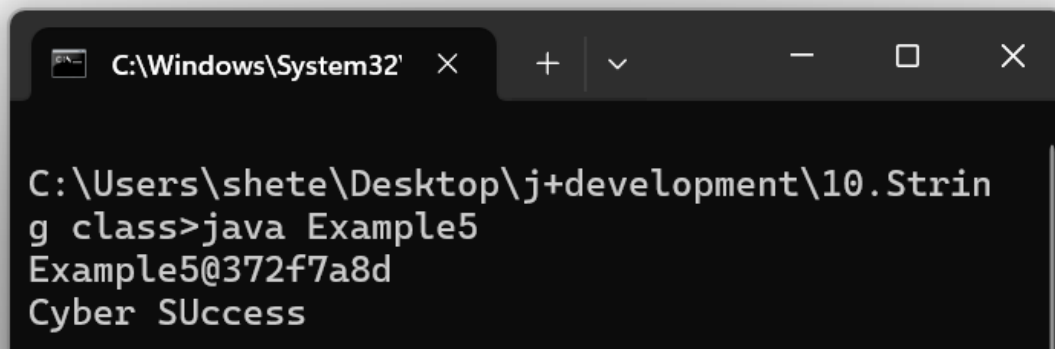


The screenshot shows a Windows command prompt window with the title 'C:\Windows\System32'. The command 'C:\Users\shete\Desktop\j+de g class>java Example6' has been executed. The output displayed is 'Example6@372f7a8d' on the first line and 'Cyber' on the second line.

7. String Classes

Can we override the toString() method of object class >>

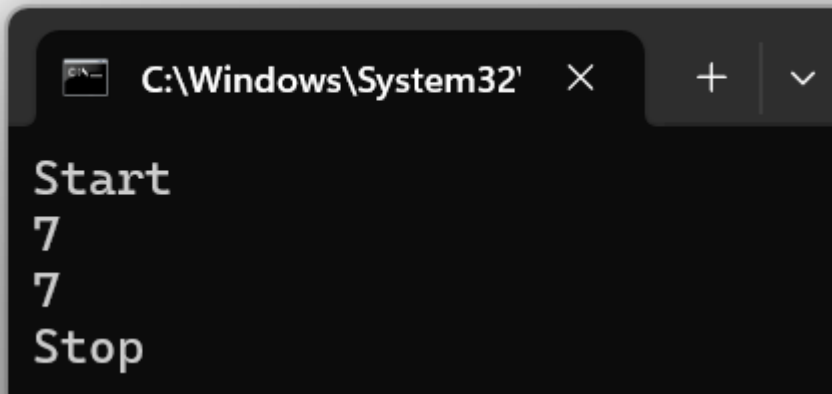
```
/*class Object
{   public String toString();
}*/
class String1
{
    public String toString()
    {
        return "Cyber SUccess";
    }
}
class Example5
{
    public static void main (String [] args)
    {
        Example5 ref = new Example5();//child
        System.out.println(ref.toString());//object class > toString() > address
        String1 str= new String1();//child
        System.out.println(str.toString());//string1 class > toString() > value
    }
}
```



7. String Classes

toString()

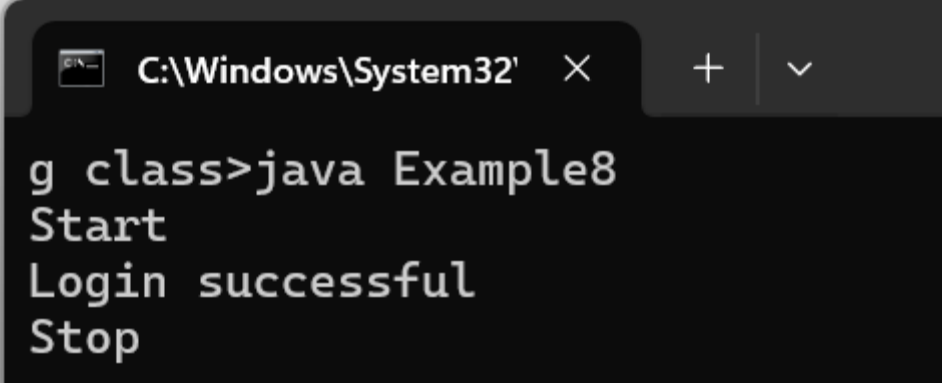
```
class Example7
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str = "Success";
        //Approach1
        int len1 = str.length();
        System.out.println(len1);
        //Approach2
        System.out.println(str.length());
        System.out.println("Stop");
    }
}
```



7. String Classes

Write a java program to check if the length of the username is more than 8 characters, if it is more than eight then display login successful else login unsuccessful.

```
class Example8
{
    public static void main(String [] args)
    {
        System.out.println("Start");
        String un = "CyberSuccess";
        if(un.length()>8)
        {
            System.out.println("Login successful");
        }
        else
        {
            System.out.println("Login unsuccessful");
        }
        System.out.println("Stop");
    }
}
```



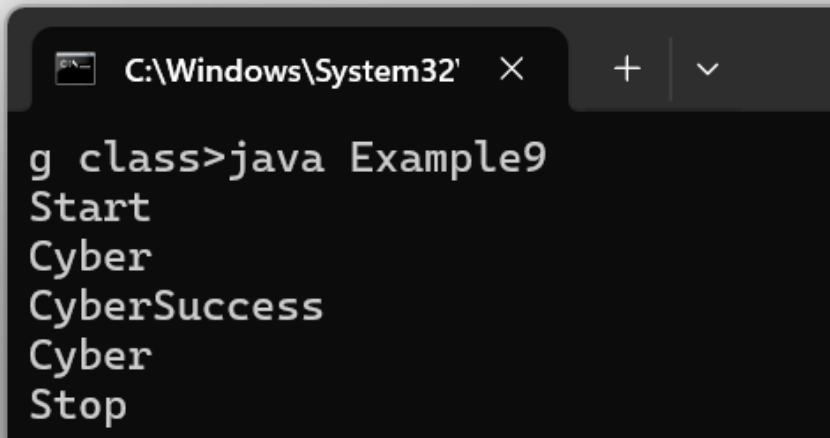
The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example8' has been entered and executed. The output of the program is displayed on separate lines: 'Start', 'Login successful', and 'Stop'.

```
g class>java Example8
Start
Login successful
Stop
```

7. String Classes

- String classes by nature called as immutable classes
- Immutable means its object data cannot go and change such a behaviour is called as immutable behaviour.

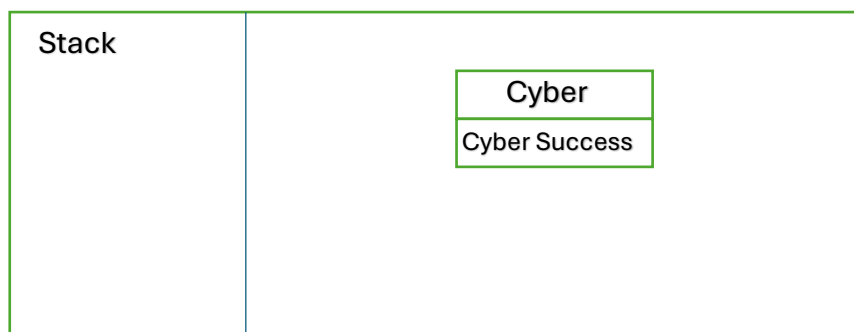
```
class Example9
{
    public static void main(String [] args)
    {
        System.out.println("Start");
        String str = "Cyber";
        System.out.println(str);
        System.out.println(str.concat("Success"));
        System.out.println(str);
        System.out.println("Stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example9' has been entered. The output of the program is displayed line by line: 'Start', 'Cyber', 'CyberSuccess', 'Cyber', and 'Stop'.

```
g class>java Example9
Start
Cyber
CyberSuccess
Cyber
Stop
```

7. String Classes



Whenever we try to store concatenated object within the string it will not change its original object value but we create a new object and will point to new object such a behaviour is called as immutable behaviour.

```
class Example10
{
    public static void main(String [] args)
    {
        System.out.println("Start");
        String str = "Cyber";
        System.out.println(str); //cyber
        String str1 = str.concat("Success"); //success
        System.out.println(str1); //cyberSuccess
        System.out.println("Stop");
    }
}
```

The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example10' has been entered. The output of the program is displayed as follows:

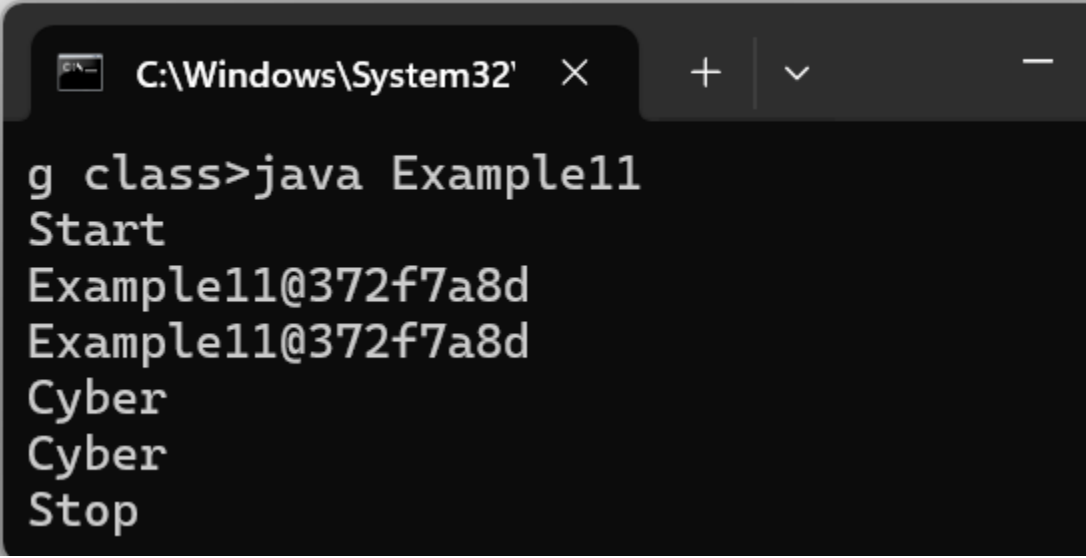
```
Start
Cyber
CyberSuccess
Stop
```

7. String Classes

StringBuffer class

StringBuffer is also part of java.lang package.

```
class Example11
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        Example11 ref = new Example11();
        System.out.println(ref);
        System.out.println(ref.toString());
        //StringBuffer sb1= "Cyber";invalid
        StringBuffer sb1 = new StringBuffer("Cyber");
        System.out.println(sb1);
        System.out.println(sb1.toString());
        System.out.println("Stop");
    }
}
//public StringBuffer(String);
```



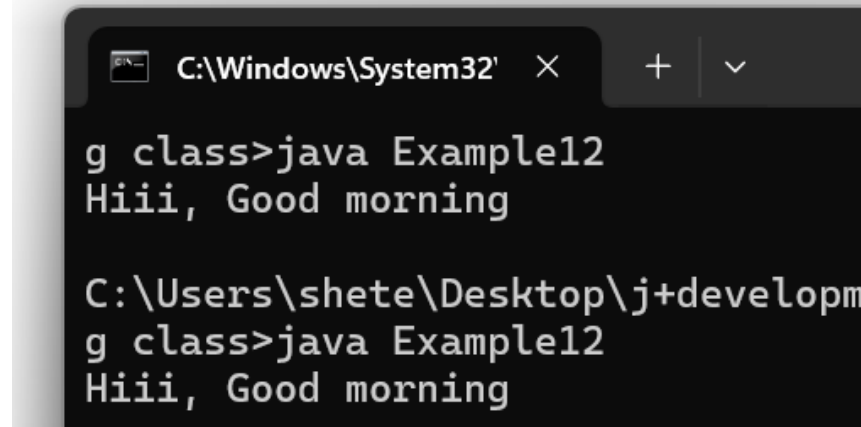
The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example11' has been entered. The output of the program is displayed line by line: 'Start', 'Example11@372f7a8d', 'Example11@372f7a8d', 'Cyber', 'Cyber', and 'Stop'.

```
g class>java Example11
Start
Example11@372f7a8d
Example11@372f7a8d
Cyber
Cyber
Stop
```

7. String Classes

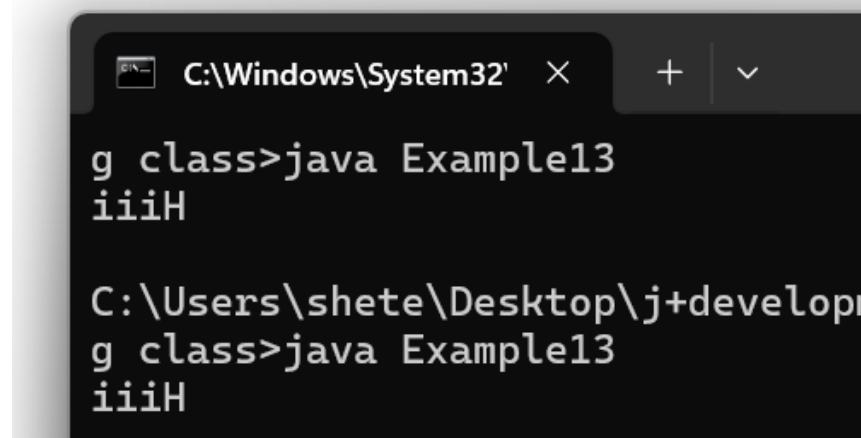
String Buffer class is called as mutable class as its object data can go and change.

```
class Example12
{
    public static void main (String [] args)
    {
        StringBuffer str= new StringBuffer("Hiii");
        str.append(", Good morning");
        System.out.println(str);
    }
}
//StringBuffer append(String);
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example12' is entered, and the output 'Hiii, Good morning' is displayed. The command is then repeated, and the same output is shown again.

```
class Example13
{
    public static void main (String [] args)
    {
        StringBuffer str= new StringBuffer("Hiii");
        str.reverse();
        System.out.println(str);
    }
}
//StringBuffer reverse();
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'g class>java Example13' is entered, and the output 'iiiH' is displayed. The command is then repeated, and the same output is shown again.

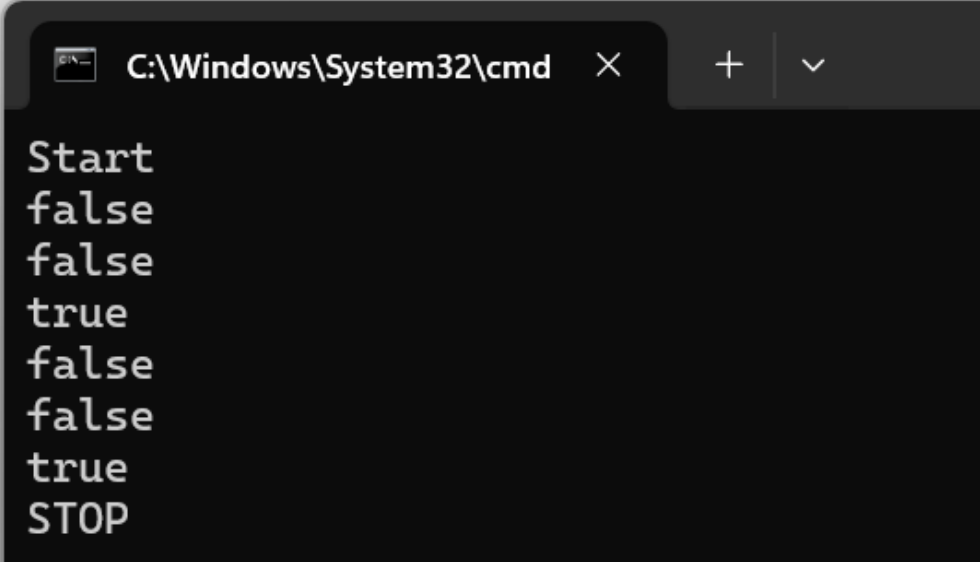
7. String Classes

If we have to compare the content of both the Strings then what approach will you take.

We use equals method.

```
class Example14
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str1="Cyber";
        String str2="Success";
        String str3="Cyber";
        System.out.println(str1.equals(str2));
        System.out.println(str2.equals(str3));
        System.out.println(str3.equals(str1));
        String s1 = new String ("Cyber");
        String s2 = new String ("Success");
        String s3 = new String ("Cyber");
        System.out.println(str1.equals(str2));
        System.out.println(str2.equals(str3));
        System.out.println(str3.equals(str1));
        System.out.println("STOP");
    }
}

//comparison >> Reference >> ==
//Comparison >> content >> equals
//public boolean equals(Object);
```



The screenshot shows a Windows Command Prompt window with the title bar 'C:\Windows\System32\cmd'. The output of the Java program is displayed as follows:

```
Start
false
false
true
false
false
true
STOP
```

- == performs reference comparison
- Equals() performs content comparison
- Equals() in string buffer doesn't perform content comparison but perform reference comparison

7. String Classes

Write difference between == operator and equals()

- The == operator will always perform reference comparison.
- Whereas equals method performs both reference comparison as well as content comparison.

When does it perform reference comparison as well as content comparison

- If the equals method is called from string class it will perform content comparison.
- But if the same equals() is called from the string buffer class it will perform reference comparison.

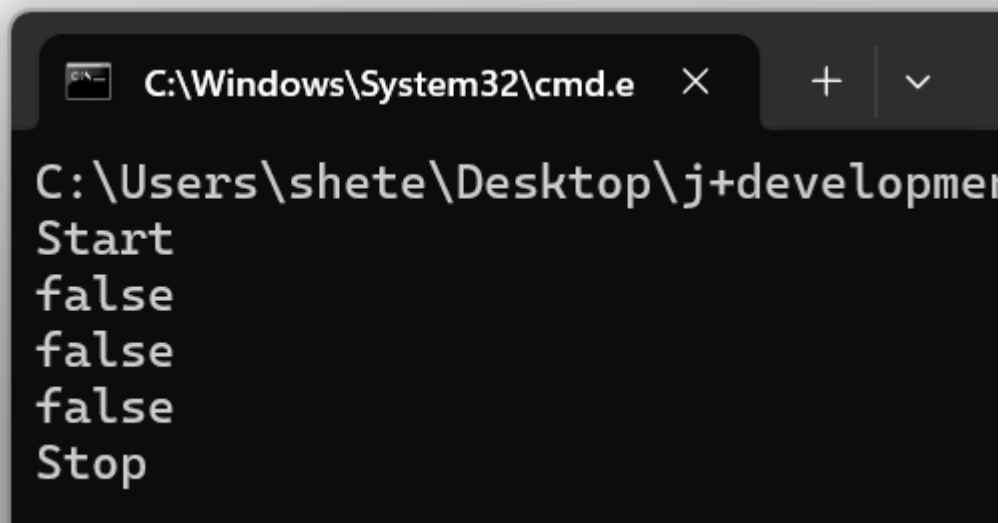
Reason:

Equals() always perform reference comparison but in String class

Equals() is overridden which was inherited from object class.

in String Buffer equals() is not overridden that's why it performs reference comparison.

```
class Example15
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        StringBuffer sb1 = new StringBuffer("Cyber");
        StringBuffer sb2 = new StringBuffer("Success");
        StringBuffer sb3 = new StringBuffer("Cyber");
        System.out.println(sb1.equals(sb2));
        System.out.println(sb2.equals(sb3));
        System.out.println(sb3.equals(sb1));
        System.out.println("Stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32\cmd.e'. The command prompt is running a Java program, and the output is displayed as follows:

```
C:\Users\shete\Desktop\j+development>
Start
false
false
false
Stop
```

7. String Classes

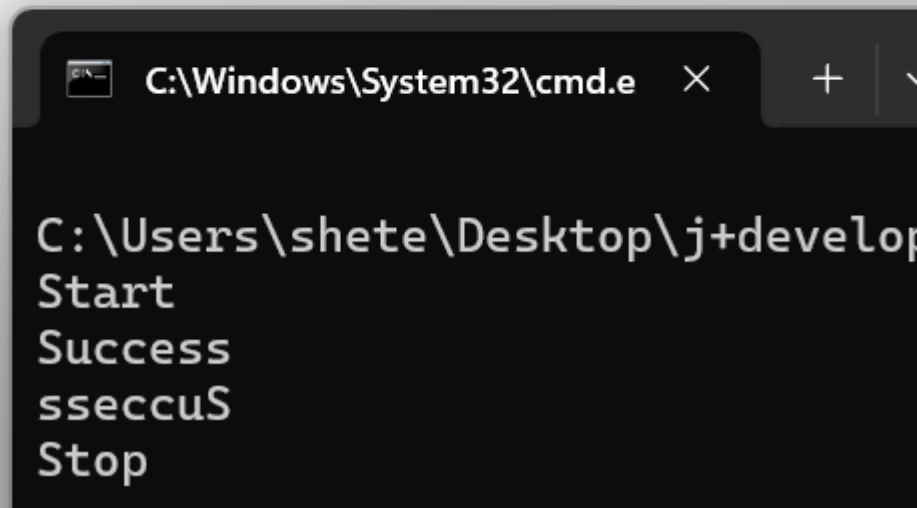
Interview Question

How do we convert string to string buffer and string buffer to string.

1.String to string Buffer

To convert string to string Buffer we can use constructor based approach.

```
class Example16
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str = "Success";
        StringBuffer sb1 = new StringBuffer(str);
        System.out.println(sb1);
        System.out.println(sb1.reverse());
        System.out.println("Stop");
    }
}
```



7. String Classes

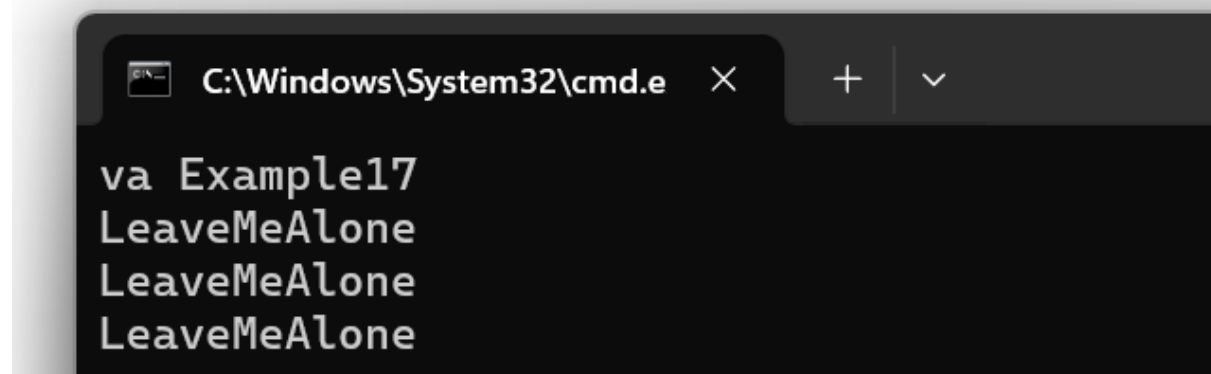
2.String buffer to String

There are two approaches to convert String buffer to String

1.Constructor based approach

2. toString()

```
class Example17
{
    public static void main (String [] args)
    {
        StringBuffer sb1 = new StringBuffer("LeaveMeAlone");
        System.out.println(sb1);
        String str = new String(sb1);
        System.out.println(str); //approach1>constructor
        //System.out.println(str.reverse());
        String str2 = sb1.toString(); //approach2>toString();
        System.out.println(str2);
        //System.out.println(str2.reverse());
    }
}
//String Class >> public String(StringBuffer)
//StringBuffer >> public String toString();
```



```
C:\Windows\System32\cmd.e X + v
va Example17
LeaveMeAlone
LeaveMeAlone
LeaveMeAlone
```

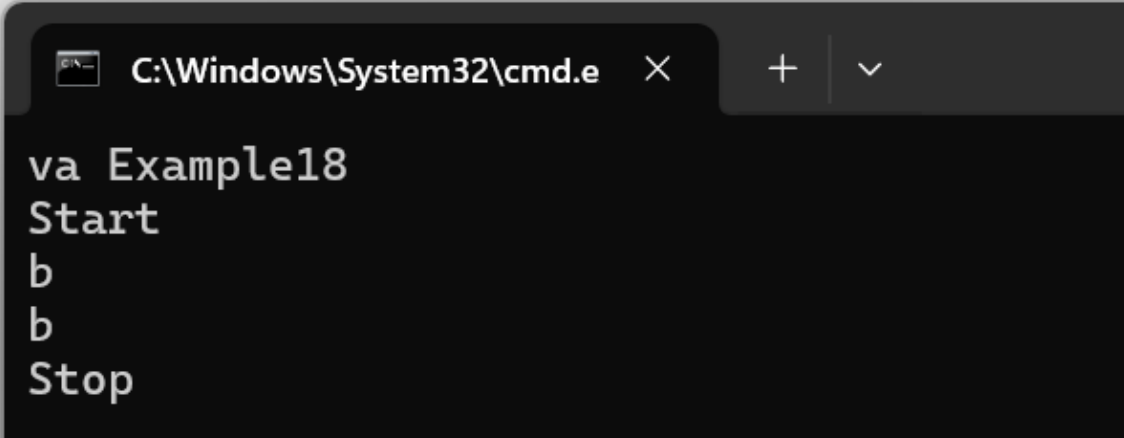
7. String Classes

charAt()

The charAt() will accept an index arguments and will return the specified character at the respective position.

If the index is not present it will generate an exception. (Error)

```
class Example18
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str = "Cyber";
        System.out.println(str.charAt(2));
        System.out.println(str.charAt(2)); //ExceptionRaised
        System.out.println("Stop");
    }
} //public char charAt(int index)
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32\cmd.e'. The command 'va Example18' has been entered, and the output is displayed as follows:

```
va Example18
Start
b
b
Stop
```

7. String Classes

Contains();

```
public Boolean contains(CharSequence s);
```

The contains method will check if a character sequence is present within the String, if the character sequence is present, it will return true else it will return false.

isEmpty()

```
public Boolean isEmpty();
```

The isEmpty method will check if the String is empty or not, if it is empty it will return true else it will return as false.

startsWith()

```
public Boolean startsWith(String prefix)
```

The startsWith method will also check whether the starting sequence of the String is matching or not, if matching will return true else return false.

endsWith(String suffix);

```
public Boolean endsWith(String suffix)
```

The endsWith method will check character sequence at the end of sequence, if the character sequence is matching or not, if matching will return true else return false.

```
//example19
```

7. String Classes

IndexOf()

Public int indexOf(String str)

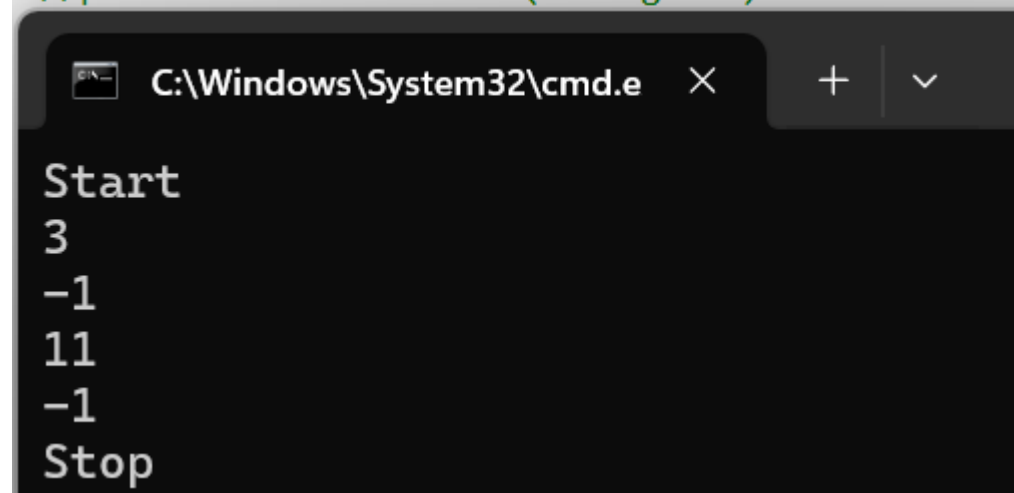
The indexOf method will return the index of the first occurrence of respective character (String format), if the respective letter or String is not present it will return -1.

lastIndexOf()

public int lastIndexOf()

The lastIndexOf method will accept the character in string format and will return an int index of the last occurrence of respective character(String format), if the respective letter of String is not present it will return -1.

```
class Example20
{
    public static void main(String [] args)
    {
        System.out.println("Start");
        String str= "CyberSuccess";
        System.out.println(str.indexOf("e"));
        System.out.println(str.indexOf("z"));
        System.out.println(str.lastIndexOf("s"));
        System.out.println(str.lastIndexOf("z"));
        System.out.println("Stop");
    }
}
//public int indexOf(String str)
//public int lastIndexOf(String str)
```

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' and standard window controls. The command prompt displays the output of the Java program: 'Start' on the first line, followed by the values '3', '-1', '11', and '-1' on the next four lines, and 'Stop' on the final line. These values correspond to the results of the indexOf and lastIndexOf operations on the string 'CyberSuccess'.

//String in built method are frequently asked in interviews.

7. String Classes

SubString

A subString can be defined as an extraction of part of string, itself is called as SubString.

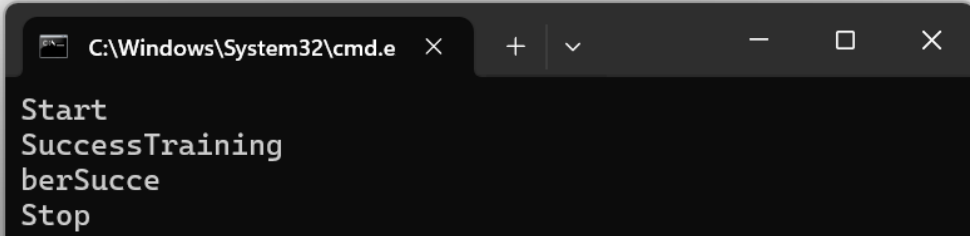
1.Public string subString(int beginIndex)

The substring method will accept the begin index, from that index onwards it will start extracting in the end of the string, if the begin index is not present it will raise an exception.

2.Public string subString(int beginIndex, int endIndex)

The substring method will start extracting from begin index and will keep on extracting till end index -1

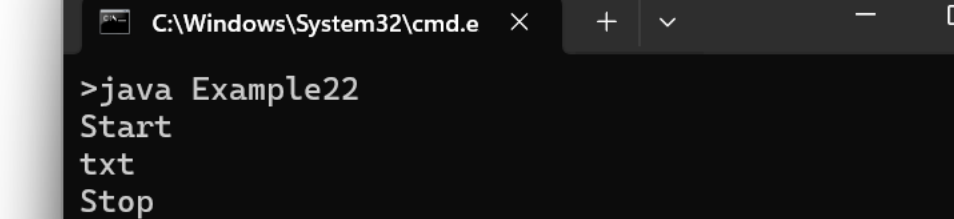
```
class Example21
{
    public static void main(String [] args)
    {
        System.out.println("Start");
        String str= "CyberSuccessTraining";
        System.out.println(str.substring(5));
        //System.out.println(str.substring(30)); //StringIndexOutOfBoundsException
        System.out.println(str.substring(2,10));
        //System.out.println(str.substring(4,90)); //StringIndexOutOfBoundsException
        System.out.println("Stop");
    }
}
//public String substring(int beginIndex)
//public String substring(int beginIndex,int endIndex)
```



```
C:\Windows\System32\cmd.e  x  +  v  -  □  x

Start
SuccessTraining
berSucce
Stop
```

```
class Example22
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String fileName="CyberSUccess.txt";
        //int a= fileName.indexOf(".");
        //System.out.println(a);
        //System.out.println(fileName.substring(a+1));
        System.out.println(fileName.substring(fileName.indexOf(".") +1));
        System.out.println("Stop");
    }
}
```



```
C:\Windows\System32\cmd.e  x  +  v  -  □

>java Example22
Start
txt
Stop
```

7. String Classes

Public string toUpperCase()

Public string toLowerCase()

The toUpperCase and toLowerCase method are used convert the string either to the uppercase or in the lowercase.

Public String trim()

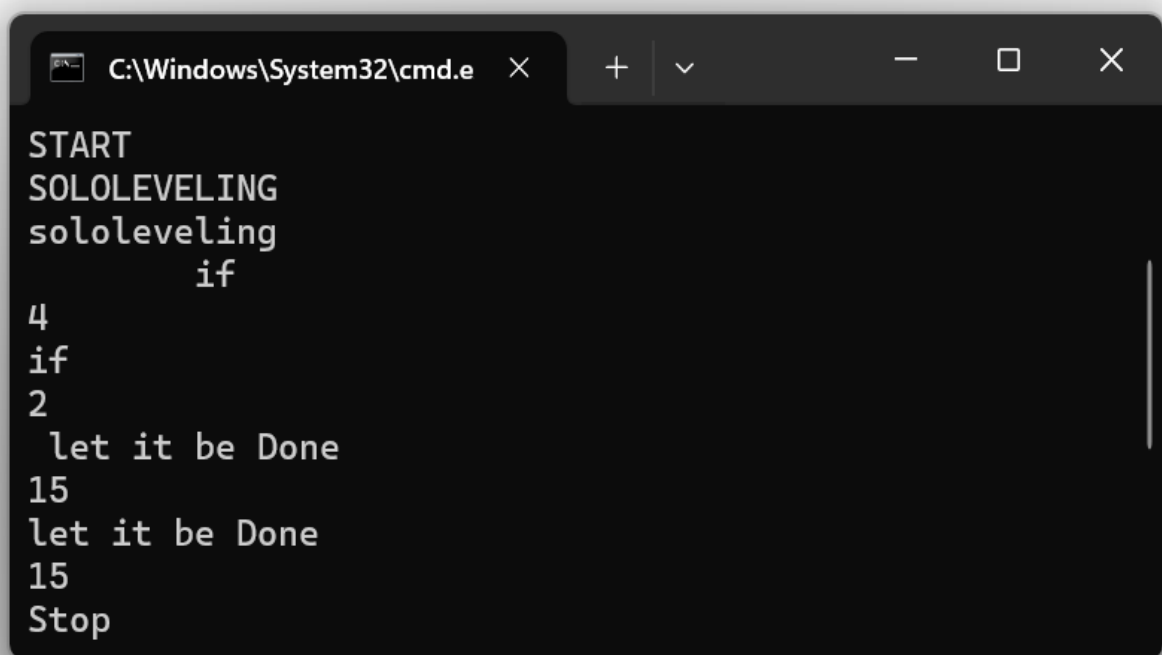
The trim method is responsible to trim the spaces on either sides of the strings the trim method cannot trim the spaces between the strings.

..example23

```
class Example23
{
    public static void main(String [] args)
    {
        System.out.println("START");
        String str1="Sololeveling";
        System.out.println(str1.toUpperCase());
        System.out.println(str1.toLowerCase());

        String s1=" if "; System.out.println(s1);System.out.println(s1.length());
        System.out.println(s1.trim());
        String z1=s1.trim();System.out.println(z1.length());

        String s2=" let it be Done";System.out.println(s2);System.out.println(s2.length());
        System.out.println(s2.trim());
        String z2=s2.trim();System.out.println(s2.length());
        System.out.println("Stop");
    }
}
//public String toUpperCase()
//public String toLowerCase()
//public String trim()
```



The screenshot shows a Windows Command Prompt window with the title bar 'C:\Windows\System32\cmd.e'. The output of the Java program is displayed as follows:

```
START
SOLELEVELING
sololeveling
    if
4
if
2
    let it be Done
15
let it be Done
15
Stop
```

7. String Classes

Public String[] split(String regex)

It returns array

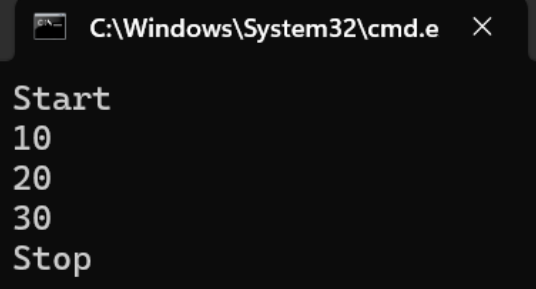
If we want to store multiple values then we need to go for an array, whenever we store multiple values in array the index will always start from zero.

Type[] name = {};

Int [] a={};

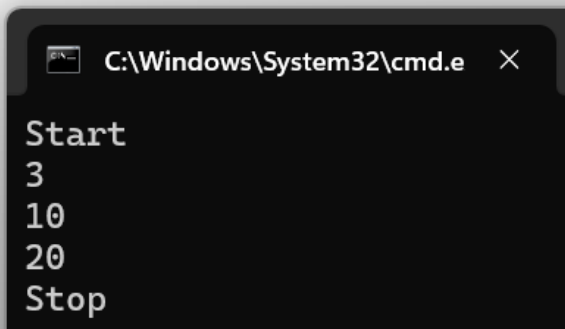
- To get a size of an array we need to use length
- Length in an array is treated as a variable.
- Whenever we get length of an array, the length of array will always start from 1.

```
class Example24
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        int[] a={10,20,30};
        System.out.println(a[0]);
        System.out.println(a[1]);
        System.out.println(a[2]);
        System.out.println("Stop");
    }
}
```



```
C:\Windows\System32\cmd.e  X
Start
10
20
30
Stop
```

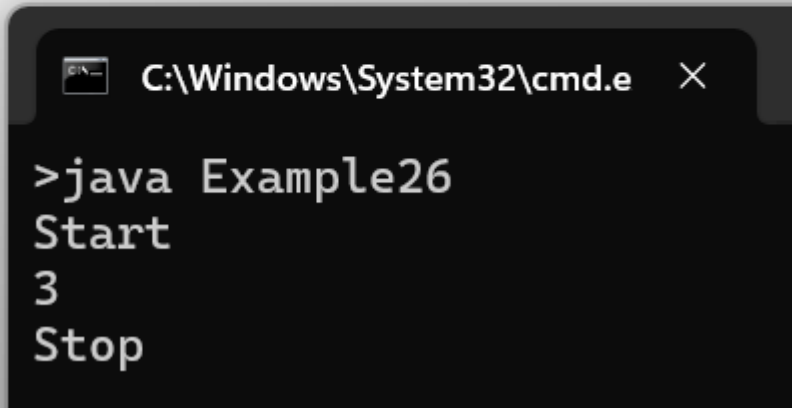
```
class Example25
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        int[] a={10,20,30};
        System.out.println(a.length);
        for(int i=0;i<2;i++)
        {
            System.out.println(a[i]);
        }
        System.out.println("Stop");
    }
}
```



```
C:\Windows\System32\cmd.e  X
Start
3
10
20
Stop
```

7. String Classes

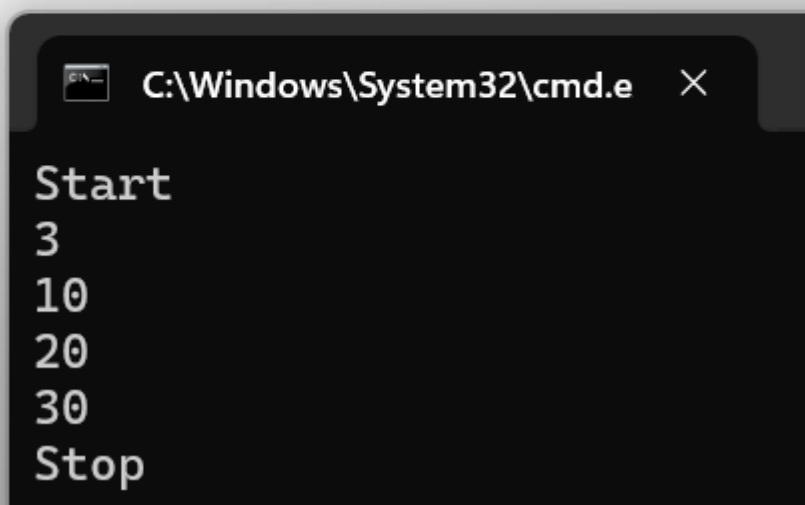
```
class Example26
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        int[] a={10,20,30};
        System.out.println(a.length);
        System.out.println("Stop");
    }
}
```



A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.e". The window shows the command ">java Example26" being executed. The output of the program is displayed as follows:

```
>java Example26
Start
3
Stop
```

```
class Example27
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        int[] a={10,20,30};
        System.out.println(a.length);
        for(int i=0;i<a.length;i++)
        {
            System.out.println(a[i]);
        }
        System.out.println("Stop");
    }
}
```



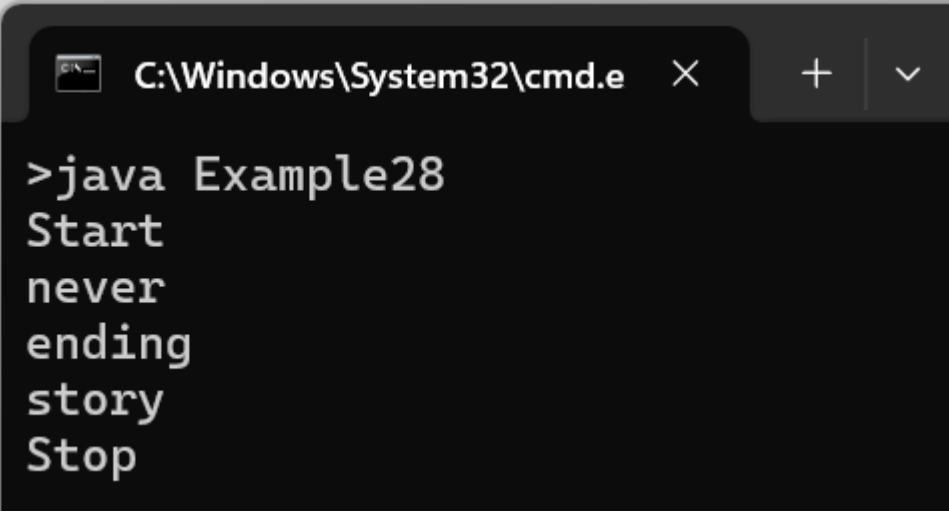
A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.e". The window shows the output of the program Example27. The output is as follows:

```
Start
3
10
20
30
Stop
```


7. String Classes

Str.split();

```
class Example28
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str="never ending story";
        String [] arr=str.split(" ");
        for(int i=0;i<arr.length;i++)
        {
            System.out.println(arr[i]);
        }
        System.out.println("Stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32\cmd.e'. The command '>java Example28' has been entered and executed. The output of the program is displayed on the following lines: 'Start', 'never', 'ending', 'story', and 'Stop'.

```
>java Example28
Start
never
ending
story
Stop
```

7. String Classes

compareTo

```
//public int compareTo(java.lang.Object);
```

The way compare to method works it will compare both the strings the way compared to works is if the both strings are exactly same it will then return zero. If both the strings are not same it will return either + or – integer

When will it return + or –

1.Less than

If the first String first letter Unicode value is less than second String first letter Unicode value it will return a negative or –value.

2.Greater than

If the first string first letter Unicode value is greater than second string first letter Unicode value it will return a positive or + value.

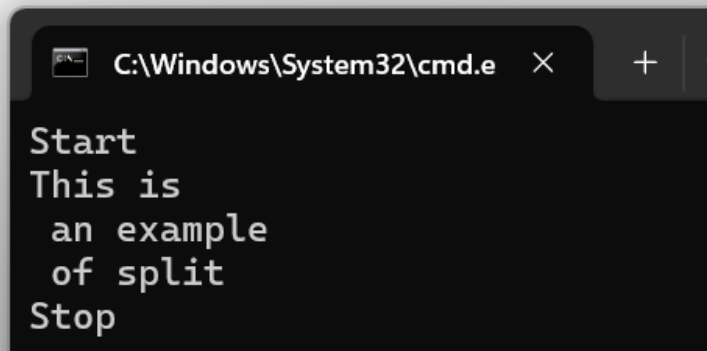
What is difference between equalsIgnoreCase() and equals().

Equals() perform case sensitive comparison where as equalsIgnoreCase() performs case insensitive comparison.

What is difference between compareToIgnoreCase() and compareTo().

compareTo() perform case sensitive comparison where as compareToIgnoreCase() performs case insensitive comparison.

```
class Example29
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str="This is, an example, of split";
        String [] arr=str.split(",");
        for(int i=0;i<arr.length;i++)
        {
            System.out.println(arr[i]);
        }
        System.out.println("Stop");
    }
}
```

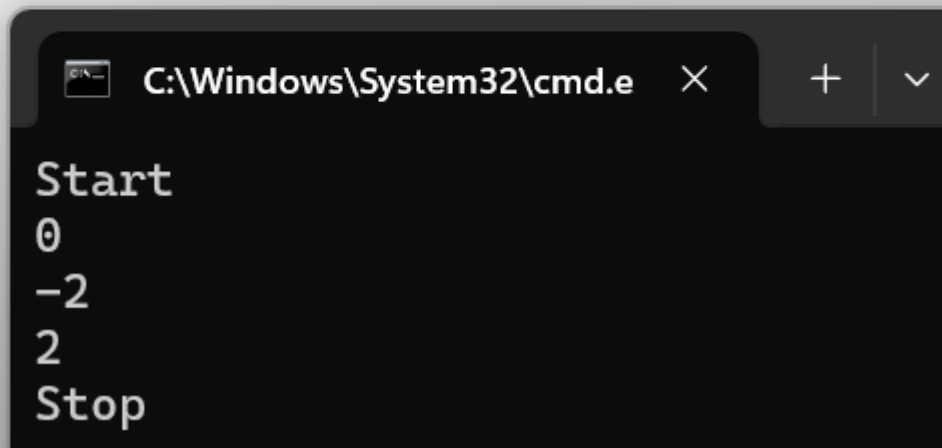


```
C:\Windows\System32\cmd.e  X  +  \

Start
This is
an example
of split
Stop
```

7. String Classes

```
class Example30
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String s1 ="Cyber",s2="Cyber";
        System.out.println(s1.compareTo(s2));
        String s11="Amber",s22="Cyber";
        System.out.println(s11.compareTo(s22));
        String s111="Cyber",s222="Amber";
        System.out.println(s111.compareTo(s222));
        System.out.println("Stop");
    }
}
```

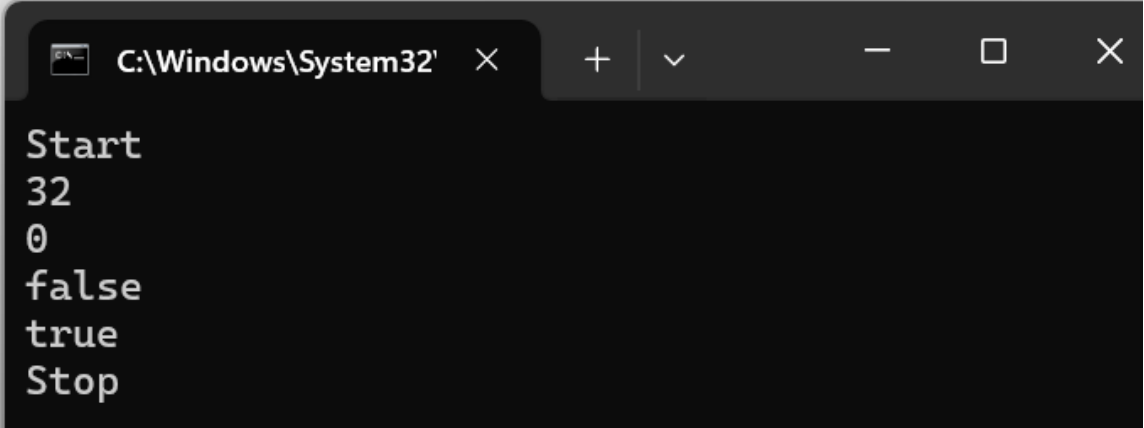


A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' with standard window controls. The command prompt displays the output of the Java program: 'Start', '0', '-2', '2', and 'Stop' on separate lines.

```
C:\Windows\System32\cmd.e
Start
0
-2
2
Stop
```

7. String Classes

```
class Example31
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str2 = "Cyber";
        String str1 = "cyber";
        System.out.println(str1.compareTo(str2));
        System.out.println(str1.compareToIgnoreCase(str2));
        System.out.println(str1.equals(str2));
        System.out.println(str1.equalsIgnoreCase(str2));
        System.out.println("Stop");
    }
}
//public boolean equalsIgnoreCase(java.lang.String)
//public int compareToIgnoreCase(java.lang.String)
```



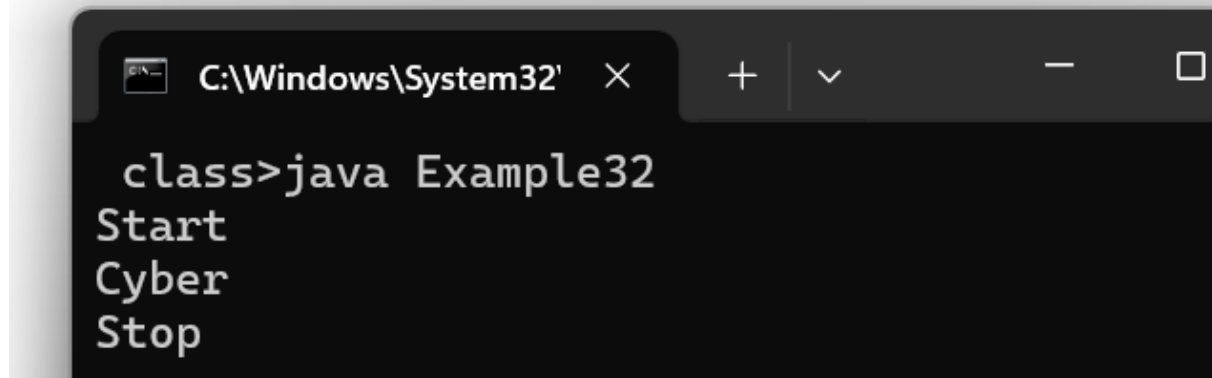
The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The window contains the following output from the Java program:

```
Start
32
0
false
true
Stop
```

3.String Builder

- String builder class is part of java.lang package.

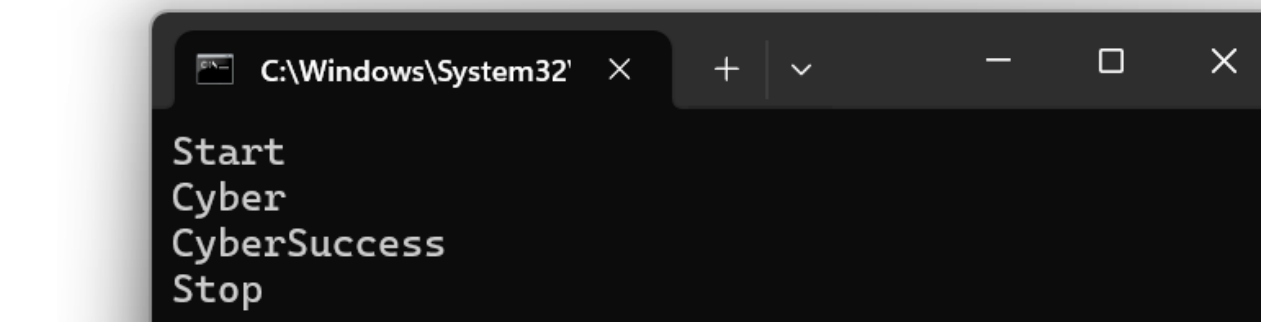
```
class Example32
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        StringBuilder sb1 = new StringBuilder("Cyber");
        System.out.println(sb1);
        System.out.println("Stop");
    }
}
//public StringBuilder(String);
```



```
C:\Windows\System32' x + v - □
class>java Example32
Start
Cyber
Stop
```

- The string builder class does not allow to create an object without new keyword it allows to create an object with new keyword.

```
class Example33
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        StringBuilder sb1 = new StringBuilder("Cyber");
        System.out.println(sb1);
        sb1.append("Success");
        System.out.println(sb1);
        System.out.println("Stop");
    }
}
//public StringBuilder(String);
```



```
C:\Windows\System32' x + v - □ x
Start
Cyber
CyberSuccess
Stop
```

- String builder by nature is mutable class.

7. String Classes

- The string buffer class and string builder class by nature are almost same. However, string buffer methods are synchronized, whereas string builder methods are non-synchronized.

What do you mean by synchronized and non-synchronized?

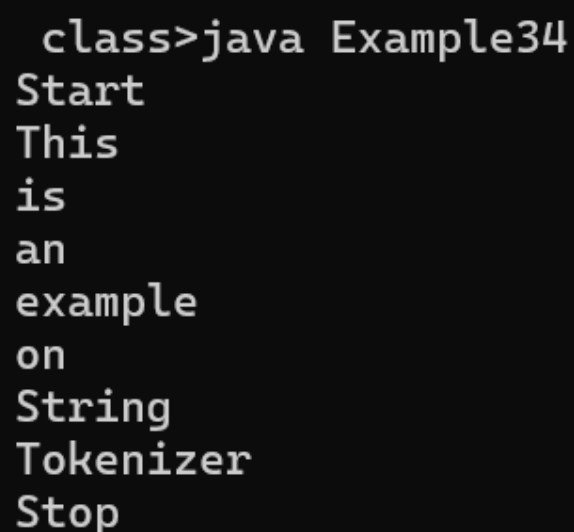
If a method is synchronized, a single thread is able to access a single method, transactions are safe, but performance is a little lower.

If the method is non-synchronized, multiple threads are able to access single method, transactions are not safe, but performance is better.

4.String Tokenizer

- The purpose of string tokenizer is to split the string
- String tokenizer is part of java.util package.
- String tokenizer by default will split the string based on spaces.
- Whenever String Tokenizer object gets created it will by default split the string with respect to spaces.
- To extract the split String we have to perform two actions
 - 1.Check if the element is present or not.
 - 2.If the element is present then extract the element.

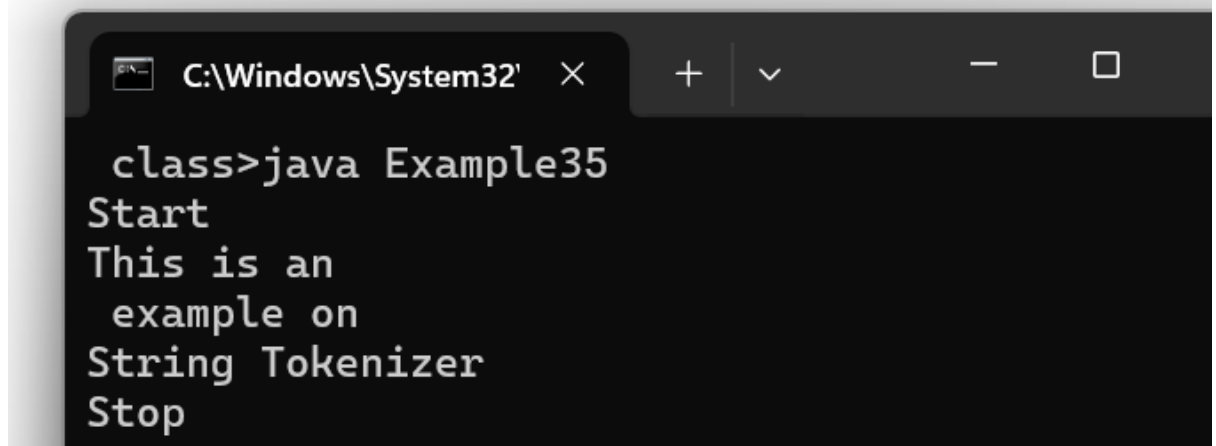
```
import java.util.StringTokenizer;
class Example34
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str = "This is an example on String Tokenizer";
        StringTokenizer st1 = new StringTokenizer(str);
        while(st1.hasMoreElements())
        {
            String c = (String)st1.nextElement();
            System.out.println(c);
        }
        System.out.println("Stop");
    }
}
//public StringTokenizer(String);
//public boolean hasMoreElements();
//public Object nextElement();
```



```
class>java Example34
Start
This
is
an
example
on
String
Tokenizer
Stop
```

7. String Classes

```
import java.util.StringTokenizer;
class Example35
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        String str = "This is an, example on ,String Tokenizer";
        StringTokenizer st1 = new StringTokenizer(str,",");
        while(st1.hasMoreElements())
        {
            String c = (String)st1.nextElement();
            System.out.println(c);
        }
        System.out.println("Stop");
    }
}
//public StringTokenizer(String str, String delim);
```



```
C:\Windows\System32' X + v - □
class>java Example35
Start
This is an
example on
String Tokenizer
Stop
```

Split method and string tokenizer both will perform the same operation but we will prefer to go for split method as string tokenizer is a legacy class and comes with number of limitations.

If I want to split the string I can directly use that object

IMP

What is String?

A string can be defined as sequence of characters or numbers or special characters within the pair of double quotes is called as String.

How many ways are there to create String object?

1. With new keyword
 2. Without new keyword
-

What type of memory do they consume?

When we create a String object without using new keyword, it uses the String Constant Pool inside heap memory. It checks whether the same String object is already present or not. If present, it returns the same reference. If not present, it creates a new object in the String Constant Pool.

When we create a String object using new keyword, a new String object is created in heap memory every time, even if the same value is already present in the String Constant Pool. It creates a separate object and returns a new reference.

The == operator will perform what comparison

- A. Reference comparison
- B. Content comparison

It will perform reference comparison in the String class.

Whenever we print a normal class reference variable it will print the address but whenever we print string class reference variable it doesn't print the address but it prints the value, Why?

- Whenever we print the reference variable of normal class it will internally call the toString() of the object class, the toString() of the object class will always be responsible to print the address.
 - But within the string class the toString() method is overridden hence rather than returning the address it returns the value.
-

7. String Classes

How can we calculate size of string?

length()

What type of comparison does equals() perform in String class?

Content comparison

Write difference between == operator and equals()

- The == operator will always perform reference comparison.
- Whereas equals method performs both reference comparison as well as content comparison.

When does it perform reference comparison as well as content comparison

- If the equals method is called from string class it will perform content comparison.
- But if the same equals() is called from the string buffer class it will perform reference comparison.

Reason:

Equals() always perform reference comparison but in String class

Equals() is overridden which was inherited from object class.

in String Buffer equals() is not overridden that's why it performs reference comparison.

How do we convert string to string buffer and string buffer to string.

1. String to string Buffer

To convert string to string Buffer we can use constructor based approach.

2. String buffer to String

There are two approaches to convert String buffer to String

1. Constructor based approach
 2. toString()
-

charAt()

The charAt() will accept an index arguments and will return the specified character at the respective position.

compareTo()

```
//public int compareTo(java.lang.Object);
```

7. String Classes

The way compare to method works it will compare both the strings the way compared to works is if the both strings are exactly same it will then return zero. If both the strings are not same it will return either + or – integer.

When will it return + or –

1. Less than

If the first String first letter Unicode value is less than second String first letter Unicode value it will return a negative or – value.

2. Greater than

If the first string first letter Unicode value is greater than second string first letter Unicode value it will return a positive or + value.

What is difference between equalsIgnoreCase() and equals().

Equals() perform case sensitive comparison where as equalsIgnoreCase() performs case insensitive comparison.

What is difference between compareToIgnoreCase() and compareTo().

compareTo() perform case sensitive comparison where as compareToIgnoreCase() performs case insensitive comparison.

Difference between String, StringBuffer and StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Memory Change	New object created on modification	Same object modified	Same object modified
Thread Safe	Yes (immutable nature)	Yes (synchronized)	No
Performance	Slow if frequent changes	Medium	Fastest
Synchronization	Not required	Yes	No
Content Comparison	equals() compares content	Use toString().equals()	Use toString().equals()

7. String Classes

Feature	String	StringBuffer	StringBuilder
Storage	SCP / Heap	Heap	Heap
Best Use	Fixed text	Multithreaded mutable text	Single-threaded mutable text
Introduced In	Java 1.0	Java 1.0	Java 1.5
Methods	concat(), substring()	append(), insert(), reverse()	append(), insert(), reverse()

Quick Memory Trick:

String → Immutable

StringBuffer → Mutable + Thread Safe(Synchronized)

StringBuilder → Mutable + Fastest (Asynchronized)

What do you mean by synchronized and non-synchronized?

If a method is synchronized a single thread is able to access a single method, transactions are safe but performance is little lower.

If the method is non-synchronized multiple threads are able to access single method transactions are not safe but performance is better.
